

Grivora AI

Production Workflow Documentation

Complete System Architecture, Methodology & Technical Reference

Data Analysis

Predictive ML

Business Intelligence

Auto Dashboard

AI Chat

Built with: Python 3.10+ | Flask | Google Gemini 2.5 Flash | scikit-learn | Chart.js | Pandas
Formats: CSV | Excel | JSON | Parquet | XML | ODS | 38 REST API Endpoints

Table of Contents

Section 1	-- Platform Overview & Vision
Section 2	-- Full System Architecture
Section 3	-- Technology Stack
Section 4	-- Directory Structure
Section 5	-- Data Ingestion Pipeline
Section 6	-- Module 01: Data Analysis (DA)
Section 7	-- Module 02: Predictive Modeling & Analysis (PMA)
Section 8	-- Module 03: Business Intelligence & Analytics (BIA)
Section 9	-- Module 04: Auto Dashboard
Section 10	-- AI Agent System & LLM Layer
Section 11	-- REST API Reference
Section 12	-- Frontend Architecture
Section 13	-- Security & Configuration
Section 14	-- Data Flow Diagrams
Section 15	-- Production Deployment Guide
Section 16	-- Error Handling Strategy
Section 17	-- Performance Considerations
Appendix A	-- File Size & Format Quick Reference
Appendix B	-- Agent Decision Logic
Appendix C	-- KPI Detection Keywords

Complete System Architecture, Methodology & Technical Reference

SECTION 1 — Platform Overview & Vision

Grivora AI is a full-stack, agentic AI data intelligence platform that transforms any raw dataset into actionable insights through three specialized analytical modules — all powered by Google Gemini 2.5 Flash as the intelligence backbone.

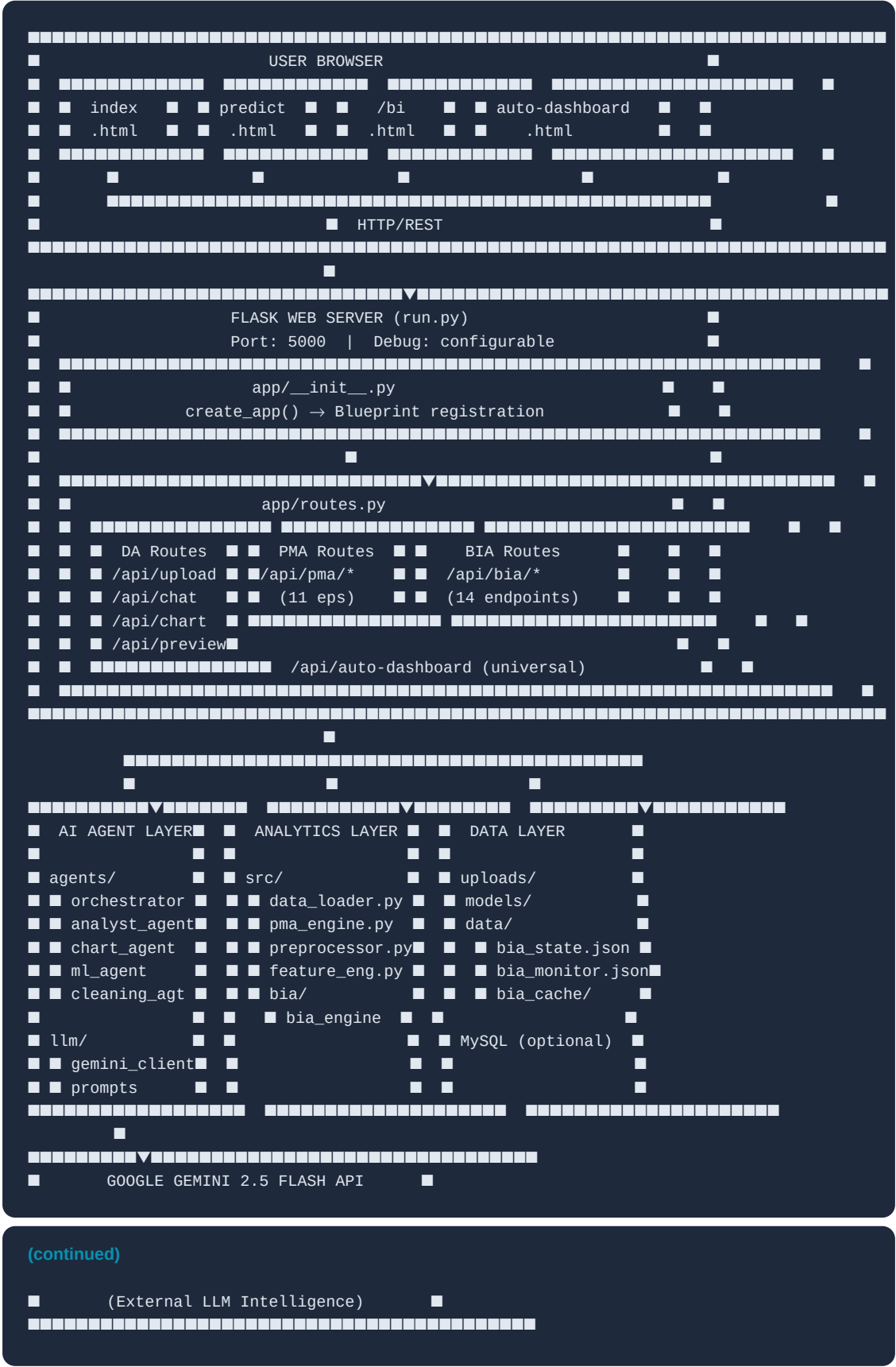
Core Mission

"Upload any data. Get instant analysis, ML predictions, BI dashboards, and AI narratives — with zero configuration"

Platform Modules

Module	Route	Status	Purpose
Data Analysis (DA)	/analysis	■ Live	Explore, clean, profile, visualize
Predictive ML (PMA)	/predict	■ Live	Train models, forecast, evaluate
Business Intelligence (BIA)	/bi	■ Live	KPIs, dashboards, insights
Auto Dashboard	/auto-dashboard	■ Live	One-click universal dashboard
AI Chat	/chat	■ Live	Natural language data queries

SECTION 2 — Full System Architecture



SECTION 3 — Technology Stack

Backend

Component	Technology	Version
Web Framework	Flask	≥2.3.0
Language	Python	3.10+
AI/LLM	Google Gemini 2.5 Flash	Latest
ML Engine	scikit-learn	≥1.3.0
Boosting	XGBoost + LightGBM	Optional
Time Series	statsmodels	Optional
Data Processing	pandas + numpy	≥2.0.0
Serialization	pickle (models) + json	stdlib

Data Loading (Universal)

Format	Library	Notes
CSV / TSV / TXT	pandas + chardet	10 encoding fallbacks
XLSX (modern Excel)	openpyxl	Primary engine
XLS (legacy Excel)	xlrd ≥2.0.1	Legacy format
XLSB (binary)	pyxlsb	Binary format
ODS (OpenDocument)	odfpy	LibreOffice format
JSON	pandas	6 orientation fallbacks
Parquet	pyarrow	Big data format
XML	pandas read_xml	Structured XML

Frontend

Component	Technology
Charts	Chart.js 4.4.0 (CDN)
Fonts	Bricolage Grotesque + Plus Jakarta Sans
CSS	Custom design system (CSS variables)
JS	Vanilla ES2020+ (no framework)
Icons	Inline SVG

Database

Component	Status
Primary (file-based)	uploads/ directory (always works)
MySQL	Optional (via mysql-connector-python + SQLAlchemy)
State storage	JSON files in data/ directory

Environment

Component	Description
Config	python-dotenv → .env file
Secret Key	SESSION_KEY (Flask sessions)
Gemini API	GEMINI_API_KEY
Upload Folder	uploads/ (default 16MB limit)

SECTION 4 — Directory Structure

```

my-data-project/
├──
│   ├── run.py                # ← Entry point: python run.py
│   ├── config.py            # App config (env vars, limits)
│   ├── .env                 # API keys & secrets (not in git)
│   ├── .gitignore
│   ├── requirements.txt      # Python dependencies
│   ├── requirements_fix.txt  # Universal data support packages
│   ├── install_deps.bat     # Windows one-click installer
│   ├──
│   ├── app/                 # Flask application package
│   │   ├── __init__.py      # create_app() factory
│   │   ├── routes.py        # ALL routes + API endpoints (1200+ lines)
│   │   ├── auth.py          # Authentication scaffold
│   │   ├── templates/       # Jinja2 HTML templates
│   │   │   ├── base.html    # Master layout (nav, theme, scripts)
│   │   │   ├── index.html   # Homepage (3-module landing)
│   │   │   ├── upload.html  # File upload interface
│   │   │   ├── analysis.html # Data Analysis landing
│   │   │   ├── data_preview.html # Full data preview + editing
│   │   │   ├── analysis_dashboard.html # Chart dashboard
│   │   │   ├── predict.html  # 10-step PMA wizard
│   │   │   ├── bi.html       # 12-step BIA studio
│   │   │   ├── auto_dashboard.html # Universal auto-dashboard
│   │   │   ├── chat.html     # AI chat interface
│   │   │   ├── dashboard.html # General dashboard
│   │   ├── static/
│   │   │   ├── style.css    # Production design system
│   │   │   ├── charts.js    # Chart utilities
│   │   ├──
│   │   ├── agents/         # AI Agent layer
│   │   │   ├── __init__.py
│   │   │   ├── base_agent.py # BaseAgent class (LLM wrapper)
│   │   │   ├── orchestrator.py # Router → decides which agent
│   │   │   ├── analyst_agent.py # Data analysis agent
│   │   │   ├── chart_agent.py # Visualization suggestion agent
│   │   │   ├── ml_agent.py # ML explanation + training advisor
│   │   │   ├── cleaning_agent.py # Data quality agent
│   │   │   ├──
│   │   │   ├── llm/        # LLM integration layer
│   │   │   │   ├── __init__.py
│   │   │   │   ├── gemini_client.py # Google Gemini API wrapper
│   │   │   │   ├── prompt_templates.py # System prompts for each agent
│   │   │   │   ├── tools.py # LLM tool definitions
│   │   │   │   ├──
│   │   │   ├── src/        # Core analytics engines
│   │   │   │   ├── data_loader.py # Universal file loader (10 formats)
│   │   │   │   ├── pma_engine.py # Predictive Modeling engine (500+ lines)
│   │   │   │   ├── preprocessor.py # Data cleaning utilities
│   │   │   │   ├── feature_engineering.py # Feature creation helpers

```

(continued)

```
■ ■■■ model.py           # Legacy model trainer
■ ■■■ predictor.py       # Legacy predictor
■ ■■■ visualizer.py      # Chart data utilities
■ ■■■ bia/
■   ■■■ __init__.py
■   ■■■ bia_engine.py    # BIA analytics engine (600+ lines)
■
■ ■■■ models/           # Saved ML model artifacts
■   ■■■ model_metadata.json # Registry of all trained models
■
■ ■■■ uploads/          # User-uploaded files
■   ■■■ (uploaded files)
■
■ ■■■ data/             # App state & monitoring
■   ■■■ bia_state.json
■   ■■■ bia_monitor.json # Event log
■   ■■■ bia_cache/
■
■ ■■■ notebooks/        # Jupyter exploration notebooks
■ ■■■ tests/            # Test suite
```

SECTION 5 — Data Ingestion Pipeline (Universal Loader)

5.1 Upload Flow

```

User selects file
  ▼
POST /api/upload
  ▼
app/routes.py → upload_file()
  ■■■ Save to uploads/ directory
  ■■■ Store path in Flask session['uploaded_file']
  ■■■ Return {message, path, filename}
  ▼
File available to ALL modules via session

```

5.2 Universal Load Strategy (src/data_loader.py)

```

load_file(file_path, nrows=None)
  ■
  ■■■ File exists? → FileNotFoundError if not
  ■
  ■■■ Detect extension (.csv/.xlsx/.json etc.)
  ■
  ■■■ CSV / TSV / TXT / DAT
  ■
  ■■■ chardet auto-detect encoding (reads 200KB)
  ■
  ■■■ Try detected encoding first
  ■
  ■■■ Fallback chain (10 encodings):
  ■     utf-8 → utf-8-sig → latin-1 → iso-8859-1
  ■     → cp1252 → windows-1252 → cp1250
  ■     → utf-16 → ascii → mac_roman → cp437
  ■     + on_bad_lines='skip' (never crash on bad rows)
  ■     + sep=None, engine='python' (auto-detect delimiter)
  ■
  ■■■ Excel (.xlsx / .xls / .xlsb / .xlsm / .ods)
  ■
  ■■■ Engine priority fallback:
  ■     .xlsx → openpyxl (primary) → xlrd
  ■     .xls  → xlrd (primary)    → openpyxl
  ■     .xlsb → pyxlsb           → openpyxl
  ■     .ods  → odf
  ■     (gives clear pip install instructions on ImportError)
  ■
  ■■■ JSON → tries 6 orientations (records/split/index/columns/values/None)
  ■■■ Parquet → pyarrow
  ■■■ XML → pandas read_xml
  ■■■ Unknown → fallback: try as CSV

```

5.3 Supported File Formats

Format	Extensions	Encoding Handling	Notes
CSV	.csv	■ Auto-detect (chardet + 10 fallbacks)	Most common
TSV	.tsv, .tab	■ Auto-detect	Tab-separated
Text	.txt, .dat	■ Auto-detect	Any delimiter
Excel Modern	.xlsx, .xlsm	openpyxl	Excel 2007+
Excel Legacy	.xls	xlrd ≥2.0.1	Excel 97-2003
Excel Binary	.xlsb	pyxlsb	Compressed Excel
OpenDocument	.ods	odfpy	LibreOffice
JSON	.json	—	6 orientations
Parquet	.parquet, .pq	—	Big data
XML	.xml	—	Structured XML

SECTION 6 — Module 01: Data Analysis (DA)

6.1 Workflow Overview

```

Upload File
  ▼
/data-preview → data_preview.html
  ■
  ■■■ Tab 1: Data Table
  ■   ■■■ Load data (load_file → preview_data API)
  ■   ■■■ Display 500 rows paginated
  ■   ■■■ Search/filter rows client-side
  ■   ■■■ Row count selector (25/50/100/500)
  ■
  ■■■ Tab 2: Data Profiling
  ■   ■■■ Column type detection (numeric/text)
  ■   ■■■ Missing value counts & percentages
  ■   ■■■ Unique value counts
  ■   ■■■ Statistical summary (mean/median/std/min/max)
  ■   ■■■ Top values for categorical columns
  ■   ■■■ Completeness score
  ■   ■■■ Duplicate row count
  ■
  ■■■ Tab 3: Data Cleaning
  ■   ■■■ Fill missing values (mean/median/mode/zero/custom)
  ■   ■■■ Drop duplicate rows
  ■   ■■■ Drop columns
  ■   ■■■ Rename columns
  ■   ■■■ Sort by column (asc/desc)
  ■   ■■■ Filter rows (pandas query syntax)
  ■
  ■■■ Tab 4: Chart Types
  ■   ■■■ Auto-detect: value_col, qty_col, date_col, cat_col
  ■   ■■■ Time series line chart (date × value)
  ■   ■■■ Bar chart (category × value, top 15)
  ■   ■■■ Histogram (value distribution, 20 bins)
  ■   ■■■ Pie/Donut chart (category shares, top 8)
  ■   ■■■ Scatter plot (qty × value, colored by rate)
  ■   ■■■ Treemap (region → person → value)
  ■   ■■■ Correlation heatmap (numeric cols × numeric cols)
  ■   ■■■ Box plot (up to 6 numeric cols)
  ■
  ■■■ Tab 5: AI Suggestions
  ■   ■■■ Gemini agent reads dataset context → suggestions
  ■
  ■■■ Tab 6: Manual Edit
  ■   ■■■ In-cell editing (save back to file)

```

6.2 Smart Column Detection

The system uses a keyword-map approach to auto-detect column roles:

```
python
KEYWORD_MAP = {
    'value':    ['sales', 'revenue', 'amount', 'price', 'profit', ...],
    'qty':      ['quantity', 'qty', 'count', 'units', 'orders', ...],
    'date':     ['date', 'time', 'year', 'month', 'period', ...],
    'category': ['category', 'type', 'product', 'class', 'brand', ...],
    'region':   ['region', 'city', 'state', 'country', ...],
    'person':   ['rep', 'employee', 'agent', 'assigned_to', ...],
    'channel':  ['channel', 'platform', 'source', 'medium', ...],
    'rate':     ['discount', 'margin', 'growth', 'conversion', ...],
}
```

This drives automatic chart selection without any user configuration.

6.3 API Endpoints — Data Analysis

Method	Endpoint	Purpose
POST	/api/upload	Save uploaded file, store in session
GET	/api/session-file	Return current session file path
POST	/api/preview-data	Load & profile dataset (500 rows)
POST	/api/edit-data	Apply cleaning actions to dataset
POST	/api/chart-data	Generate all chart data from smart detection
POST	/api/chat	Route to AI agent with dataset context

SECTION 7 — Module 02: Predictive Modeling & Analysis (PMA)

7.1 Full 10-Step Workflow

```

STEP 1: Upload / Select Data
  ■ ■■■ Reads session uploaded_file
  ■ ■■■ Lists all columns
  ■ ■■■ User selects TARGET COLUMN (what to predict)
  ■

STEP 2: Data Overview & Detection
  ■ ■■■ detect_data_type(df, target_col)
  ■ ■■■ Auto-detects: tabular / time_series / text
  ■ ■■■ Auto-detects: regression vs classification
  ■ ■■■ Shows: n_rows, n_cols, num_cols, cat_cols, date_cols
  ■

STEP 3: Feature Analysis & Selection
  ■ ■■■ preprocess_tabular() → X, y
  ■ ■■■ SelectKBest (f_classif / f_regression)
  ■ ■■■ Ranked feature importance bar chart
  ■

STEP 4: Data Splitting
  ■ ■■■ Interactive slider: test split (10%-40%)
  ■ ■■■ Visual train/test bar
  ■ ■■■ Optional: 3-fold or 5-fold cross-validation
  ■

STEP 5: Model Selection
  ■ ■■■ get_model_catalog(data_type, problem_type)
  ■ ■■■ recommend_models() → top 3 with reasoning
  ■ ■■■ Gemini explains why each model is recommended
  ■ ■■■ User can override and pick any model
  ■

STEP 6: Model Training Studio
  ■ ■■■ Config Panel: Epochs, Batch Size, LR, Activation, Optimizer
  ■ ■■■ Gemini AI advisor → suggests optimal config
  ■ ■■■ 8-step Pipeline Checklist (battery-charging animation)
  ■ ■ 1. Data Validation
  ■ ■ 2. Preprocessing (encode/scale/impute)
  ■ ■ 3. Feature Preparation
  ■ ■ 4. Train/Test Split
  ■ ■ 5. Model Initialization
  ■ ■ 6. Training Loop
  ■ ■ 7. Validation
  ■ ■ 8. Save Artifacts
  ■ ■■■ Live Epoch Progress Bar (simulated + real metrics)
  ■ ■■■ macOS-style Terminal Log
  ■ ■■■ Success Banner with metrics
  ■

STEP 7: Model Evaluation
  ■ ■■■ Classification: accuracy, precision, recall, F1, AUC, confusion matrix
  ■ ■■■ Regression: RMSE, MAE, R2, MAPE
  ■ ■■■ Actual vs Predicted chart (scatter or bar)
  ■ ■■■ Residuals chart (bar)
  ■ ■■■ Gemini explains metrics in plain English
  ■

```

(continued)

STEP 8: Hyperparameter Tuning

- RandomizedSearchCV (10 trials, fast)
- GridSearchCV (exhaustive, slower)
- Predefined param grids for 10 model types
- Retrains with best params
- Shows before/after metric comparison

STEP 9: Make Prediction

- Auto-generates input form from feature names
- Encodes categorical inputs via saved LabelEncoders
- Returns prediction + confidence + class probabilities
- Gemini explains the prediction result

STEP 10: Export & Deploy

- Export predictions as CSV
- Download model .pkl file path
- Gemini improvement suggestions
- Saved models registry

7.2 Model Catalog

Tabular Regression

Model	Key	Category
Linear Regression	linear_regression	Linear
Ridge Regression	ridge	Linear
Lasso Regression	lasso	Linear
ElasticNet	elasticnet	Linear
Decision Tree	decision_tree	Tree
Random Forest	random_forest	Ensemble
Gradient Boosting	gradient_boosting	Boosting
Extra Trees	extra_trees	Ensemble
Support Vector Regression	svr	SVM
XGBoost*	xgboost	Boosting
LightGBM*	lightgbm	Boosting

Tabular Classification

Model	Key	Category
Logistic Regression	logistic_regression	Linear
K-Nearest Neighbors	knn	Instance
Gaussian Naive Bayes	naive_bayes	Probabilistic

Support Vector Machine	svm	SVM
Decision Tree	decision_tree	Tree
Random Forest	random_forest	Ensemble
Gradient Boosting	gradient_boosting	Boosting
Extra Trees	extra_trees	Ensemble
XGBoost*	xgboost	Boosting
LightGBM*	lightgbm	Boosting

Time Series

Model	Notes
Random Forest	With lag features (1–5 periods)
Gradient Boosting	With calendar features
XGBoost	With lag + rolling features
ARIMA (5,1,0)	Classical statsmodels

*If package installed

7.3 Preprocessing Pipeline

```
Raw DataFrame
  ▼
detect_data_type() → {data_type, problem_type, num_cols, cat_cols, date_cols}
  ▼
preprocess_tabular()
  ■■■ Drop date columns from features (used in TS lag generation)
  ■■■ LabelEncoder on each categorical column (saved per-column)
  ■■■ Median imputation for numeric missing values
  ■■■ Mode imputation for categorical missing values
  ■■■ LabelEncoder on target if classification + string dtype
  ▼
train_test_split(test_size=user_defined, random_state=42)
  ▼
Model.fit(X_train, y_train) → Model.predict(X_test)
  ▼
compute_metrics() → save_model_artifacts() + save encoders (.pkl)
```

7.4 Model Persistence

```
models/
■■■ pma_random_forest_20241201_143022.pkl      # Trained model
■■■ pma_random_forest_20241201_143022_encoders.pkl # LabelEncoders
■■■ model_metadata.json                      # Registry (last 10 models)

model_metadata.json entry:
{
  "model_key": "random_forest",
  "model_name": "Random Forest",
  "target_col": "Sales_Amount",
  "feature_names": ["Region", "Product", "Quantity"],
  "problem_type": "regression",
  "data_type": "tabular",
  "metrics": {"rmse": 245.3, "r2": 0.87},
  "model_path": "models/pma_random_forest_20241201_143022.pkl",
  "saved_at": "20241201_143022"
}
```

7.5 API Endpoints — PMA

Method	Endpoint	Purpose
POST	/api/pma/detect	Detect data type + problem type
POST	/api/pma/columns	List all columns with types
POST	/api/pma/feature-importance	Compute SelectKBest scores
POST	/api/pma/models	Get model catalog + AI recommendations
POST	/api/pma/train	Train selected model, save artifacts
POST	/api/pma/tune	Hyperparameter tuning (grid/random)
POST	/api/pma/predict	Run prediction on new input
GET	/api/pma/saved-models	List all saved models
POST	/api/pma/load-model	Load model into session
POST	/api/pma/suggestions	Get AI improvement suggestions
POST	/api/pma/export	Export predictions as CSV

SECTION 8 — Module 03: Business Intelligence & Analytics (BIA)

8.1 Full 12-Step Workflow

STEP 1: Data Integration

- ■■■ File upload (session or new)
- ■■■ MySQL connection test (optional)
- ■■■ test_mysql_connection(host, port, user, password, database)
- ■■■ ETL Pipeline visualization (4-step animated checklist)
- 1. Extract → 2. Transform → 3. Load → 4. Index

STEP 2: ETL & Storage

- ■■■ transform_data(df, options)
- ■■■ Drop duplicates
- ■■■ Fill missing (auto/mean/median/zero)
- ■■■ Normalize numeric [0,1] (optional)
- ■■■ Auto-parse date columns
- ■■■ load_to_mysql(df, cfg, table) via SQLAlchemy

STEP 3: KPI Processing Engine

- ■■■ compute_kpis(df)
- ■■■ Auto-detect value_col, qty_col, date_col, cat_col
- ■■■ total_value, avg_value, max_value, min_value
- ■■■ period_growth (first half vs second half)
- ■■■ total_records, total_columns
- ■■■ total_qty, unique_categories
- ■■■ Format: currency / percent / number
- ■■■ aggregate_data(df, date_col, value_col)
- ■■■ Daily aggregation
- ■■■ Weekly aggregation
- ■■■ Monthly aggregation (ME)
- ■■■ Quarterly aggregation (QE)
- ■■■ Yearly aggregation (YE)

STEP 4: Exploratory Data Analysis

- ■■■ compute_eda(df)
- ■■■ Per numeric column: mean, median, std, min, max, Q1, Q3, skew, kurtosis, outliers
- ■■■ Pearson correlation matrix (up to 10 numeric cols)
- ■■■ Category value distributions (top 10 per column)
- ■■■ Dataset completeness score
- ■■■ Missing value heat counts

STEP 5: Visualization Builder

- ■■■ build_chart_data(df, chart_type, x_col, y_col, color_col, agg_func)
- ■■■ Chart types: bar, line, area, pie, scatter, histogram, heatmap
- ■■■ Aggregation: sum, mean, count, max, min
- ■■■ Color-by: optional grouping column

STEP 6: Executive Dashboard

- ■■■ /api/bia/dashboard → all-in-one call
- ■■■ KPI cards grid (all detected KPIs)
- ■■■ Time series trend chart
- ■■■ Top categories bar chart
- ■■■ Share breakdown donut chart

(continued)

- ■■■ Value distribution histogram
- ■■■ Auto-detected insights panel
-

STEP 7: Advanced Analytics

- ■■■ customer_segmentation(df, n_clusters)
- ■ ■■■ StandardScaler on numeric columns
- ■ ■■■ KMeans(n_clusters, n_init=10)
- ■ ■■■ PCA(n_components=2) for scatter visualization
- ■ ■■■ Per-cluster stats (size, mean of top 4 cols)
- ■■■ detect_anomalies(df, value_col)
- ■ ■■■ IQR-based: $Q1 - 1.5 \times IQR$, $Q3 + 1.5 \times IQR$
- ■ ■■■ Returns anomaly count, %, bounds, values
- ■■■ time_series_forecast(df, date_col, value_col, periods)
- ■■■ Monthly resample → linear trend (np.polyfit)
- ■■■ Forecast N future periods
-

STEP 8: AI Insights

- ■■■ generate_auto_insights(df, kpi_data) – rule-based
- ■ ■■■ Growth trend (positive/negative)
- ■ ■■■ Top performer (category with most value)
- ■ ■■■ Missing data warning (>5% threshold)
- ■ ■■■ Outlier detection alert
- ■ ■■■ Large dataset notice (>100K rows)
- ■ ■■■ Strong correlation alert (>0.5 Pearson)
- ■■■ Gemini LLM (optional): 3 actionable business insights
-

STEP 9: Ask Your Data

- ■■■ /api/bia/ask → passes question to Orchestrator
- ■■■ Orchestrator routes to AnalystAgent
- ■■■ Full dataset context (shape, columns, stats, sample) in prompt
-

STEP 10: Real-Time Analytics

- ■■■ Auto-refresh KPI cards (5s / 15s / 30s / 60s)
- ■■■ /api/bia/refresh → recomputes KPIs on each call
- ■■■ Live elapsed timer display
-

STEP 11: Export & Reports

- ■■■ export_to_csv(df, filename) → uploads/ directory
- ■■■ build_pdf_report(kpis, insights)
- ■ ■■■ Primary: reportlab PDF generation
- ■ ■■■ Fallback: plain .txt report
- ■■■ Download links served from uploads/
-

STEP 12: Monitoring

- ■■■ log_event(event_type, details) → data/bia_monitor.json
- ■■■ get_monitor_logs(50) → last 50 events
- ■■■ get_system_health()
 - ■■■ Disk usage (total/used/free GB)
 - ■■■ MySQL availability flag
 - ■■■ scikit-learn availability flag

8.2 API Endpoints — BIA

Method	Endpoint	Purpose
POST	/api/bia/extract	ETL Extract: profile file
POST	/api/bia/mysql-test	Test MySQL connection

POST	/api/bia/mysql-tables	List MySQL tables
POST	/api/bia/transform	Clean + normalize data
POST	/api/bia/mysql-load	Load DataFrame to MySQL
POST	/api/bia/kpis	Compute all KPIs + aggregations
POST	/api/bia/eda	Full EDA statistics
POST	/api/bia/chart	Build specific chart data
POST	/api/bia/dashboard	All-in-one dashboard data
POST	/api/bia/segment	K-Means customer segmentation
POST	/api/bia/anomalies	IQR anomaly detection
POST	/api/bia/forecast	Linear trend forecasting
POST	/api/bia/insights	AI + rule-based insights
POST	/api/bia/ask	NL query → Orchestrator
POST	/api/bia/refresh	Real-time KPI refresh
POST	/api/bia/export-csv	Export dataset as CSV
POST	/api/bia/export-report	Generate PDF/TXT report
GET	/api/bia/monitor	System logs + health
POST	/api/bia/columns	Column type listing

SECTION 9 — Module 04: Auto Dashboard (Data → Dashboard)

9.1 Single-Call Architecture

The Auto Dashboard is a **zero-config, one-API-call** dashboard generator. A single POST to `/api/auto-dashboard` returns everything needed to render a full multi-section dashboard.

```
POST /api/auto-dashboard
┌
└
  api_auto_dashboard()
  ┌
  │   Load file from session (universal load_file)
  │   ┌
  │   │   1. KPI Computation
  │   │   ┌   bia_engine.compute_kpis(df)
  │   │   │   + fundamental stats always included:
  │   │   │   total_rows, total_cols, completeness%, duplicates
  │   │   │   └
  │   │   │   2. Column Profiles (ALL columns)
  │   │   │   ┌   Per numeric: min, max, mean, median, std
  │   │   │   │   Per categorical: unique count, top value, top count
  │   │   │   └
  │   │   │   3. Chart Data Generation
  │   │   │   ┌   time_series (if date + value detected)
  │   │   │   │   bar (top 15 categories × value)
  │   │   │   │   pie/donut (top 8 categories)
  │   │   │   │   histogram (20-bin distribution)
  │   │   │   │   scatter (top 2 numeric cols, 500 sample)
  │   │   │   │   heatmap (Pearson correlation, up to 8 cols)
  │   │   │   └
  │   │   │   4. Per-Numeric Stats Charts
  │   │   │   ┌   Up to 10 numeric columns: [min, Q1≈, median, mean, Q3≈, max]
  │   │   │   └
  │   │   │   5. Category Distribution Charts
  │   │   │   ┌   Up to 8 categorical cols: top 8 value counts each
  │   │   │   └
  │   │   │   6. Extra Segment Bars
  │   │   │   ┌   value_col grouped by each additional cat_col
  │   │   │   └
  │   │   │   7. AI Insights
  │   │   │   ┌   bia_engine.generate_auto_insights(df, kpi_result)
  │   │   │   └
  │   │   │   8. Preview Rows
  │   │   │   ┌   First 10 rows, JSON-safe (NaN→null)
  │   │   │   └
  │   │   └
  │   └
  └
    Returns single JSON payload → frontend renders:
    ┌   KPI Cards Grid (animated, color-coded)
    ┌   Core Visualization Row 1 (time_series + bar)
    ┌   Core Visualization Row 2 (pie + histogram)
    ┌   Category Distribution Charts Grid
    ┌   Segment Analysis Bars Grid
    ┌   Numeric Stats Charts Grid
    ┌   Correlation Heatmap
    ┌   AI Insights Panel
    ┌   Column Profiles Grid
```

(continued)

Full Data Preview Table

9.2 Dashboard Sections (maximally informative)

Section	Charts/Panels	Scales With Data
KPI Cards	Up to 8+ cards	More columns → more KPIs
Core Charts	2–4 charts	Based on detected column types
Category Breakdowns	1 chart per cat_col	Up to 8 cols
Segment Analysis	1 bar per extra cat_col	Up to 3 extra segments
Numeric Stats	1 chart per num_col	Up to 10 columns
Heatmap	1 heatmap	Up to 8×8 matrix
Insights	N insight cards	Rule-based, data-driven
Column Profiles	1 card per column	ALL columns shown
Preview Table	10 rows × up to 50 cols	Handles wide data

SECTION 10 — AI Agent System & LLM Layer

10.1 Agent Architecture

```

User Question + Dataset
  ▼
Orchestrator.run(message, file_path)
  ■
  ■■■ 1. get_file_context(file_path)
  ■     ■■■ load_file(file_path)
  ■     ■■■ Build context string:
  ■         - Shape (rows × cols)
  ■         - Column names
  ■         - Data types
  ■         - Missing value counts
  ■         - First 10 rows
  ■         - Describe() statistics
  ■
  ■■■ 2. decide_agent(message) → Gemini LLM call
  ■     Prompt: "Choose one of: analyst, chart, ml, cleaning"
  ■     Returns: agent name
  ■
  ■■■ 3. agent.run(message, context) → Gemini LLM call
  ■     Full prompt = system_prompt + context + user_message

```

10.2 Agent Types & Responsibilities

Agent	Class	Triggers	Gemini Prompt Focus
analyst	AnalystAgent	stats, summary, columns, compare	Expert data analyst — answer from REAL dataset
chart	ChartAgent	visualize, plot, graph, chart	Visualization expert — suggest + code Plotly charts
ml	MLAgent	predict, model, forecast, train	ML engineer — explain models, metrics, improvements
cleaning	CleaningAgent	missing, duplicates, fix, quality	Data quality expert — find issues, provide pandas code

10.3 MLAgent Extended Capabilities

The MLAgent goes beyond the base agent with specialized methods:

python

```
MLAgent.explain_model_selection(data_info, recommendations)
    → "Why these 3 models are best for YOUR dataset"

MLAgent.explain_metrics(metrics, problem_type, model_name)
    → "RMSE of 245 means each prediction is ±$245 off – this is GOOD for this dataset"

MLAgent.suggest_improvements(metrics, model_key, data_info)
    → "3 specific improvements: 1. Engineer X feature, 2. Scale Y column, 3. Tune Z param"
```

10.4 Gemini Client

python

```
# llm/gemini_client.py
GeminiClient:
    model = genai.GenerativeModel("gemini-2.5-flash")

    generate(prompt: str) → str:
        response = model.generate_content(prompt)
        return response.text
```

10.5 System Prompts Design

Each agent has a carefully engineered system prompt that:

1. Establishes expert persona
2. Mandates use of REAL data (no hallucinated examples)
3. Forbids saying "please provide data" (data is always in context)
4. Sets response length/style constraints
5. Uses actual column names from the dataset

SECTION 11 — REST API Reference

11.1 Page Routes

Method	Route	Template	Purpose
GET	/	index.html	Homepage
GET	/analysis	analysis.html	DA landing
GET	/predict	predict.html	10-step PMA wizard
GET	/bi	bi.html	12-step BIA studio
GET	/auto-dashboard	auto_dashboard.html	Universal dashboard
GET	/upload	upload.html	File upload
GET	/chat	chat.html	AI chat
GET	/data-preview	data_preview.html	Data preview + edit
GET	/analysis-dashboard	analysis_dashboard.html	Chart dashboard
GET	/dashboard	dashboard.html	General dashboard

11.2 API Endpoints Summary

Category	Count	Prefix
Core / Upload	4	/api/
Data Analysis	3	/api/
PMA	11	/api/pma/
BIA	19	/api/bia/
Auto Dashboard	1	/api/
TOTAL	38	

11.3 Session State Management

Flask server-side sessions track the user journey:

python

```
session['uploaded_file']    # Current file path (shared across all modules)
session['bia_file']         # BIA-specific file path
session['bia_mysql_cfg']    # MySQL connection config
session['bia_kpi_data']     # Cached KPI computation result
session['pma_info']         # PMA data type detection result
session['pma_file']         # PMA-specific file path
session['pma_target']       # Selected target column
session['pma_model_path']   # Path to saved model .pkl
session['pma_enc_path']     # Path to saved encoders .pkl
session['pma_feature_names'] # List of feature column names
session['pma_problem_type'] # 'classification' or 'regression'
```

SECTION 12 — Frontend Architecture

12.1 Design System (style.css)

CSS

```
CSS Variables (Light Mode):
--bg, --bg2, --bg3      # Background layers
--surface, --surface2    # Card/panel surfaces
--border, --border2      # Border opacity levels
--accent (#2563eb)       # Primary blue
--accent2 (#7c3aed)      # Secondary purple
--accent3 (#0891b2)      # Tertiary cyan
--success (#059669)      # Green
--warning (#d97706)      # Amber
--danger (#dc2626)       # Red
--text, --text2, --text3 # Text hierarchy
--shadow-sm/md/lg/xl     # Elevation shadows
```

Fonts:

```
Bricolage Grotesque – headings, display text
Plus Jakarta Sans  – body, UI elements
```

12.2 Navigation Component (base.html)

Navbar:

```
[Omniora Logo] [Home] [Data Analysis] [Predictive ML] [Business Intel]
[Upload Data →] [■ Data to Dashboard]
```

```
Upload Data → blue gradient, standard CTA
Data to Dashboard → green-to-cyan gradient, distinct visual
```

12.3 Theme System

javascript

```
// Dark ↔ Light mode with ripple animation
localStorage.getItem('omniora-theme') // persisted preference
triggerRipple(goingLight)              // canvas ripple effect
setDark() / setLight()                 // applies CSS class to body
```

12.4 Step-by-Step Wizard Pattern (PMA + BIA)

Both PMA and BIA use the same wizard pattern:

- **State object** (**S** / **BS**) holds all data across steps
- **goStep(n)** / **showPage(id)** handles navigation
- **Sidebar or progress bar** shows completion

- **API calls on step entry** (lazy loading, only when needed)
 - **markDone(step)** marks steps complete with ✓ indicator
-

SECTION 13 — Security & Configuration

13.1 Environment Variables (.env)

```
GEMINI_API_KEY=your_gemini_api_key_here
SECRET_KEY=your_flask_secret_key_here
UPLOAD_FOLDER=uploads
DEBUG=True
```

13.2 Security Settings

Setting	Value	Purpose
MAX_CONTENT_LENGTH	16 MB	Prevent large file uploads
SESSION_TYPE	Filesystem	Flask session storage
SECRET_KEY	From .env	Session signing
File storage	uploads/ dir	Isolated upload zone

13.3 File Safety

- Files saved to **uploads/** directory (sandboxed)
- Only pandas-readable formats processed
- No execution of uploaded content
- Flask session used for state (not URL params)

SECTION 14 — Data Flow Diagrams

14.1 Complete User Journey



(continued)

```
■ POST /api/bia/forecast    → predictions
■ POST /api/bia/insights   → AI insights
■ POST /api/bia/ask        → NL query
■ POST /api/bia/refresh    → live KPIs
■ POST /api/bia/export-csv → CSV download
■ GET  /api/bia/monitor    → system health
```

14.2 AI Intelligence Flow

```
Any User Question
  ▼
Orchestrator.decide_agent(message)
  ▼ [Gemini LLM call #1: "Which agent?"]
  ■
  ■ ■ ■ "analyst" → AnalystAgent (stats questions)
  ■ ■ ■ "chart"  → ChartAgent  (visualization requests)
  ■ ■ ■ "ml"     → MLAgent    (prediction/model questions)
  ■ ■ ■ "cleaning" → CleaningAgent (data quality questions)
  ▼
  get_file_context(file_path)
  ▼ [Build context: shape + dtypes + missing + 10 rows + describe()]
  ▼
  agent.run(message, context)
  ▼ [Gemini LLM call #2: "Answer with real data context"]
  ▼
  return response → user
```


SECTION 15 — Production Deployment Guide

15.1 Local Development

bash

```
# 1. Clone / navigate to project
cd my-data-project

# 2. Create virtual environment
python -m venv .venv
.venv\Scripts\activate      # Windows
# source .venv/bin/activate # Linux/Mac

# 3. Install all dependencies
pip install -r requirements_fix.txt

# 4. Set up environment
copy .env.example .env      # Edit with your GEMINI_API_KEY

# 5. Run development server
python run.py
# Server starts at: http://localhost:5000
```

15.2 Environment Configuration

```
.env file:
GEMINI_API_KEY=Aiza...
SECRET_KEY=generate_a_random_32char_string_here
UPLOAD_FOLDER=uploads
DEBUG=False      # Set to False for production
```

15.3 Required pip installs (quick reference)

bash

```
# Core
pip install pandas numpy flask python-dotenv

# Excel support (CRITICAL – fixes all Excel errors)
pip install openpyxl xlrd>=2.0.1 pyxlsb odfpy

# Encoding detection (CRITICAL – fixes UTF-8 errors)
pip install chardet

# AI
pip install google-generativeai

# ML
pip install scikit-learn xgboost lightgbm statsmodels

# Reporting
pip install reportlab pyarrow

# Database (optional)
pip install mysql-connector-python sqlalchemy
```

15.4 Production Server (Gunicorn)

bash

```
pip install gunicorn
gunicorn -w 4 -b 0.0.0.0:8000 "app:create_app()"
```

15.5 Nginx Reverse Proxy Config

nginx

```
server {
    listen 80;
    server_name yourdomain.com;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        client_max_body_size 32M; # Match Flask limit
    }

    location /uploads/ {
        alias /path/to/project/uploads/; # Serve files directly
    }
}
```

SECTION 16 — Error Handling Strategy

16.1 Universal Data Loader Errors

Error Type	Root Cause	Handling
UnicodeDecodeError	CSV with Windows encoding (0x92 etc.)	10-encoding fallback chain
ImportError (xlrd)	.xls without xlrd>=2.0.1	Try openpyxl, show pip command
ImportError (openpyxl)	.xlsx without openpyxl	Show pip command
EmptyDataError	Zero-row file	Caught, returns descriptive error
ParserError	Malformed CSV	on_bad_lines='skip' handles it
FileNotFoundError	Session expired / file deleted	Returns clear error message

16.2 API Error Responses

All API endpoints follow this pattern:

python

```
try:
    # ... processing ...
    return jsonify({...data...})
except Exception as e:
    return jsonify({'error': str(e)}), 500
```

Frontend handles:

javascript

```
if (data.error) {
    show error message to user
    return // Stop processing
}
// Continue with data
```

16.3 Common Error Messages → Solutions

Error	Solution
'Import xlrd' failed	<code>pip install xlrd>=2.0.1</code>
'utf-8' codec can't decode byte 0x92	<code>pip install chardet</code> (auto-detect)
No file in session	Upload a file first via <code>/upload</code>
No model trained yet	Complete training step in <code>/predict</code>

Need at least 2 numeric columns	Dataset needs more numeric data
GEMINI_API_KEY not found	Add key to .env file

SECTION 17 — Performance Considerations

17.1 Large Data Handling

Technique	Where Used	Benefit
nrows=10,000 preview	bia_engine.extract_file	Fast preview of large files
head(500) for table display	preview_data API	Limits JSON payload
head(10) for chart samples	auto_dashboard	Fast rendering
sample(min(500, n)) for scatter	chart engines	Cap scatter points
num_cols[:8] for heatmap	EDA engine	Limit correlation matrix
cat_cols[:8] for distributions	BIA engine	Limit cat breakdowns
records[:500] for clustering	segmentation	Cap PCA inputs

17.2 Session-Based Caching

python

```
session['bia_kpi_data']    # KPI result cached per session
session['pma_info']        # Data type detection cached
session['pma_feature_names'] # Feature names cached after training
```

17.3 Chart.js Client-Side Rendering

All charts render **client-side** using Chart.js 4.4.0 (CDN):

- Server sends pure JSON data (labels + values arrays)
- No server-side image generation (scales to any screen)
- Responsive by default
- Destroy + recreate pattern prevents canvas memory leaks

17.4 Recommended Data Size Limits

Operation	Recommended Max	Notes
File upload	16 MB (configurable)	Flask MAX_CONTENT_LENGTH
Data preview	500 rows shown	Full data in file
Chart data	20 bars, 8 pie slices	Top-N truncation
Scatter plot	500 points	Random sample
Heatmap	8×8 matrix	Top N numeric cols

Clustering	500 points visualized	Full data used for fit
ML training	~500K rows tested	Depends on model

APPENDIX A — File Size & Format Quick Reference

Format	Extension	Encoding Handling	Engine
CSV	.csv	chardet + 10 fallbacks	pandas
TSV	.tsv .tab	chardet + 10 fallbacks	pandas (sep=\t)
Text	.txt .dat	chardet + 10 fallbacks	pandas (auto-sep)
Excel	.xlsx	N/A (binary)	openpyxl
Excel	.xls	N/A (binary)	xlrd>=2.0.1
Excel Bin	.xlsb	N/A (binary)	pyxlsb
ODS	.ods	N/A (binary)	odfpy
JSON	.json	UTF-8	pandas (6 orient)
Parquet	.parquet	N/A (binary)	pyarrow
XML	.xml	UTF-8	pandas read_xml

APPENDIX B — Agent Decision Logic

Message Keywords → Agent Selected

"statistics", "average", "mean", "summary", "columns",
"how many", "what is", "describe", "compare"
→ AnalystAgent

"chart", "graph", "plot", "visualize", "show me",
"bar chart", "pie chart", "histogram"
→ ChartAgent

"predict", "forecast", "model", "machine learning",
"train", "accuracy", "regression", "classify"
→ MLAgent

"missing", "null", "duplicate", "clean", "fix",
"quality", "error", "invalid", "wrong"
→ CleaningAgent

APPENDIX C — KPI Detection Keywords

python

```
VALUE_KEYS = ["sales", "revenue", "amount", "total", "profit", "income",  
              "price", "cost", "value", "score", "earnings", "turnover"]  
  
QTY_KEYS    = ["quantity", "qty", "count", "units", "orders", "volume",  
              "num", "sold", "items", "pieces", "cases"]  
  
DATE_KEYS   = ["date", "time", "created", "month", "year", "period",  
              "timestamp", "updated", "posted", "day", "week"]  
  
CAT_KEYS    = ["category", "type", "product", "class", "brand", "dept",  
              "segment", "channel", "region", "status", "group", "label"]
```

Grivora AI Production Workflow Documentation

Generated: 2025 | Version 1.0 | Full Architecture Reference